

Implementación de sistema para INNOVATE SOLUTIONS

Consigna

"La empresa **"Innovate Solutions"** necesita desplegar urgentemente la primera versión de su nueva plataforma de e-commerce en un entorno de desarrollo basado en Kubernetes (Minikube, preferiblemente configurado con múltiples nodos). Se les encarga a ustedes, como equipo de DevOps, implementar la arquitectura completa de tres capas: un Frontend web estático (HTML/PHP servido por Nginx), una API Backend (Node.js/Express) y una base de datos (PostgreSQL). Deben utilizar Deployments con múltiples réplicas (al menos 4) para el frontend y backend, y un StatefulSet para la base de datos asegurando la persistencia de datos mediante PersistentVolumeClaims.

Es requisito indispensable para **"Innovate Solutions"** garantizar la resiliencia y seguir las mejores prácticas desde esta fase inicial. Por ello, deberán gestionar la configuración de la API y las credenciales de la base de datos de forma segura utilizando *ConfigMaps* y *Secrets* respectivamente. Además, implementarán livenessProbes y readinessProbes para todos los componentes, definirán requests y limits de CPU/Memoria para cada contenedor, y configurarán topologySpreadConstraints en los Deployments para simular una distribución tolerante a fallos entre diferentes "zonas" (nodos de Minikube).

El objetivo es entregar un sistema funcional, observable y preparado para futuras implementaciones (guía con los pasos realizados)."

Junto con la presente guía se entrega todo el código relacionado a la misma

Resolucion

Parte 1 - Construcción de imágenes en Docker

Se procede primero a crear las carpetas necesarias para el proyecto:

```
mkdir proyecto
cd proyecto
mkdir backend
mkdir frontend
```

Primero construiremos el código del frontend. Para esto ejecutar los siguientes comandos:

```
cd frontend
touch index.php
touch Dockerfile
```

Dentro del archivo index.php colocar el código suministrado en el Anexo I. Dentro de este Dockerfile colocar:

```
FROM php:apache
RUN apt-get update && apt-get install -y libpq-dev && docker-php-ext-install
mysql && docker-php-ext-enable mysql
COPY index.php /var/www/html
```

Continuar con:

```
cd ..
cd backend
touch package.json
touch server.js
touch Dockerfile
```

Copiar dentro del archivo server.js y dentro del archivo package.json el código suministrado en el Anexo II. Luego, dentro del Dockerfile en esta carpeta agregar:

```
FROM node:20
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Ahora se procede a la construcción de las imágenes, Antes de esto, ingresar en consola:

```
cd ..  
minikube start
```

Esperar a que termine la inicialización, luego:

```
eval $(minikube docker-env)
```

A partir de ahora los comandos de docker serán ejecutados dentro del entorno de minikube. Hacer entonces:

```
docker build -t frontend-app ./frontend  
docker build -t backend-app ./backend
```

Esperar que cada una de las imágenes se construya.

Parte 2 - Configuración de Kubernetes y minikube

A partir de este momento las imagenes ya estan construidas, es momento de configurar todo nuestro entorno de kubernetes, para esto, crear desde consola los siguientes archivos:

```
01-namespace.yml  
02-secret.yml  
03-configmap.yml  
04-postgres-service-headless.yml  
05-postgres-service-clusterip.yml  
06-postgres-statefulset.yml  
07-backend-deployment.yml  
08-backend-service.yml  
09-frontend-deployment.yml  
10-frontend-service.yml
```

El contenido de cada uno de los archivos se encuentra en el anexo III.

Luego de haber creado cada uno de los archivos e insertar el contenido indicado, ejecutar todos los comandos que siguen a continuación en el orden indicado:

```
kubectl apply -f 01-namespace.yml  
kubectl apply -f 02-secret.yml -n innovate-ecommerce  
kubectl apply -f 03-configmap.yml -n innovate-ecommerce  
kubectl apply -f 04-postgres-service-headless.yml -n innovate-ecommerce  
kubectl apply -f 05-postgres-service-clusterip.yml -n innovate-ecommerce  
kubectl apply -f 06-postgres-statefulset.yml -n innovate-ecommerce
```

```
kubectl apply -f 07-backend-deployment.yml -n innovate-commerce
kubectl apply -f 08-backend-service.yml -n innovate-commerce
kubectl apply -f 09-frontend-deployment.yml -n innovate-commerce
kubectl apply -f 10-frontend-service.yml -n innovate-commerce
```

Uno de los últimos pasos a ejecutar consiste en agregar una tabla a nuestra base de datos, para esto utilizar el siguiente comando:

```
kubectl exec -it postgres-db-0 -n innovate-commerce -- psql -U postgres -d ecommerce -c "
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
"
```

La razón del mismo es que nos permite ejecutar comandos de postgres dentro de nuestro pod.

A partir de este momento, todos los servicios de nuestra aplicación deberían encontrarse funcionando, podemos observar el estado de todos nuestros pods ejecutando el siguiente comando por consola

```
kubectl get pods -n innovate-commerce
```

Y podemos además conocer el estado de todos nuestros servicios

```
kubectl get service -n innovate-commerce
```

Para observar el frontend de nuestro proyecto ejecutamos:

```
minikube service frontend-service -n innovate-commerce
```